

RAG / LLM Çalışma Notları

Retrieval-Augmented Generation · Embedding · Vektör Arama · Değerlendirme

Sayfa 1	Embedding & Vektör Arama
Sayfa 2	Chunking & Parsing
Sayfa 3	Retrieval Kalitesi
Sayfa 4	LLM & GenAI Katmanı
Sayfa 5	Değerlendirme (Evaluation)
Sayfa 6	Production'a Taşıma

Her sayfa aynı iskelete sahip:

- ① **Bu nedir** — kavramların tanımı
 - ② **Nasıl yapılır** — işin akışı ve mantığı
 - ③ **Karar noktaları** — hangi durumda hangi seçenek
 - ④ **Tuzaklar** — nerede sessizce bozulur
- + **Kontrol yöntemi** — doğru çalıştığını nasıl kanıtlarım
- + **Mülakat soruları** — sözlü anlatarak test et

Kesikli çerçeveli "GÖZLEM" kutuları ezberlenmek için değil — kuralların neden böyle olduğunu gösteren gerçek ölçümlerdir.

Embedding & Vektör Arama

① Bu nedir

- **Embedding:** metni sayı vektörüne çevirmek. Yakın anlam = yakın vektör.
- **Vektör uzayı:** bu vektörlerin yaşadığı N boyutlu uzay. Arama = "en yakın komşuyu bul".
- **Vektör veritabanı:** vektörleri saklayıp "en yakını bul" sorgusunu hızlı cevaplayan sistem. Örnek: Qdrant, Milvus, pgvector.
- **FAISS:** veritabanı **değil**, arama kütüphanesi. Kalıcılık, filtreleme, çok kullanıcı desteği yok.
- **Index:** vektörlerin arama için düzenlenmiş hali. Tipi, hızı ve kesinliği belirler.
- **ANN (Approximate Nearest Neighbor):** yaklaşık komşu arama. Kesinlikten az feragat, çok hız kazancı.
- **Bi-encoder:** sorgu ve doküman **ayrı** vektörlenir → hızlı → **retrieval** için.
- **Cross-encoder:** sorgu ve doküman **birlikte** modele girer, tek skor çıkar → yavaş ama kaliteli → **rerank** için.
- **Normalize:** vektörü birim uzunluğa getirmek.
- **Cosine:** iki vektör arasındaki açı benzerliği. Normalize edilmişse dot product'a eşittir.

② Nasıl yapılır

Doküman → parçala (chunk) → her chunk'ı embedding'e çevir → index'e yaz
Soru → aynı modelle embedding'e çevir → en yakın N chunk'ı bul → LLM'e ver

Kritik: soru ve dokümanlar **aynı modelle** vektörlenmek zorunda. Farklı model = anlamsız sonuç.

Arama iki aşamalıdır

- **1. aşama — geniş tara:** embedding ile hızlıca 20–50 aday bul. Amaç: doğru cevabı kaçırmamak (*recall*).
- **2. aşama — daralt:** cross-encoder bu adayları yeniden sıralar, en iyi 3–5'i seçilir (*precision*).

Neden ikisi: hızlı model kalitesiz, kaliteli model yavaş. İkisini birleştirirsin.

③ Karar noktaları

Karar	Kural
Hangi embedding modeli?	Dile göre. Türkçe içerik varsa çok dilli model (bge-m3 , multilingual-e5). Sadece İngilizce eğitilmiş model kullanma.
Chunk ne kadar olsun?	Token ile ölç, modelin limitinin %80'ini geçme.
Hangi index tipi?	<100K vektör → Flat (dokunma) · 100K+ → IVFFlat · hız kritik → HNSW · bellek dar → IVFPQ
Mesafe metriği?	Normalize et, inner product kullan.

Model seçmeden önce 5 kontrol

1. Tokenizer diyakritikleri (ç ş ğ ı ö ü) koruyor mu?
2. `max_seq_length` kaç?
3. Boyut kaç? (384 hızlı ↔ 1024 kaliteli)
4. Prefix istiyor mu? (E5 ailesi `"query:"` / `"passage:"` zorunlu kılar)
5. Kararı kendi verinde ölç. Sıralama tabloları (MTEB) sadece başlangıç noktası.

④ Tuzaklar

Tuzak	Sonuç
Chunk > model limiti	Metnin sonu embedding'e girmez, hiç aranamaz
İngilizce tokenizer + Türkçe metin	Diyakritik silinir; <code>poliçe</code> ile <code>police</code> aynı vektör olur
Normalize yok + L2 mesafesi	Uzun vektörler cezalanır, sıralama bozulur
Model değişti, index aynı	Uyumsuz vektör uzayları, anlamsız sonuç

Dördünün ortak özelliği: hata mesajı vermezler.

Kontrol yöntemi

1. Tokenizer benim kelimelerimi nasıl parçalıyor? → token listesini gözle oku.
2. Anlamı yakın iki cümlelerin benzerlik skoru gerçekten yüksek mi? → ölç, tahmin etme.

Mülakat soruları

1. Retrieval neden iki aşamalı? Bi-encoder ve cross-encoder nerede kullanılır?
2. Flat yerine ne zaman IVFFlat / HNSW kullanırsın?
3. Embedding modelini değiştirdin — index'e ne yapman gerekir, neden?
4. Cosine, dot product ve L2 ilişkisi nedir? Normalize edilmişse hangisi hangisine denk?

GÖZLEM — EZBERLEMEK İÇİN DEĞİL

all-MiniLM-L6-v2 (salt İngilizce) modelinde ölçülen gerçek değerler: $\cos(\text{"poliçe"}, \text{"police"}) = 1.0000 \rightarrow$ tokenizer ç'yi siliyor, iki kelime birebir aynı vektör. $\cos(\text{"gemi gövdesi dayanımı"}, \text{"hull strength"}) = 0.029$ ama aynı ifadenin `"yemek tarifi"` ile benzerliği **0.358** → Türkçe ifade, kendi çevirisinden 12 kat çok alakasız metne benziyor. İngilizce tarafta 0.90 vs 0.05 ile kusursuz çalışıyor. **Sonuç: bu model Türkçe'de anlam değil "dil parmak izi" ölçüyor.**

Chunking & Parsing

① Bu nedir

- **Parsing:** dosyadan metni çıkarmak (PDF/DOCX → text). Yapıyı (başlık, tablo, sıra) koruyup korumaman kaliteyi belirler.
- **Chunking:** dokümanı parçalara bölmek. Üç sebepten zorunlu: embedding modelinin token limiti var; LLM'e tüm dokümanı veremezsin; 500 sayfa tek vektöre sıkışırsa anlam erir.
- **Overlap:** komşu chunk'ların ortak kısmı. Sınırdaki bölünen cümle/fikir kaybolmasın diye.
- **Metadata:** chunk'a eklenen kaynak bilgisi (dosya adı, sayfa no). Kaynak göstermek ve filtreleme için.

② Nasıl yapılır

Dosya → parse (metin + yapı) → chunk'la (+ metadata) → embed → index

Chunking stratejileri (basitten iyiye)

1. **Fixed-size** — sabit N token. Basit ama cümleyi ortadan böler.
2. **Recursive** — önce paragraf sınırından böl; sığmazsa satır, sonra cümle, sonra kelime. Pratikte iyi varsayılan.
3. **Yapı bazlı (layout-aware)** — başlık/bölüm sınırlarından böl. Teknik dokümanda en iyisi.
4. **Semantic** — cümleleri embedding'le, anlam değiştiği yerde kes. Pahalı, bazen daha iyi.

İki güçlendirme (kaliteyi belirgin artırır)

- **Contextual retrieval:** her chunk'ın başına LLM ile 1–2 cümle bağlam ekle ("Bu bölüm X raporunun finans kısmından, Y'yi anlatıyor"), sonra embed et. Chunk tek başına okunduğunda anlamsızlaşma problemini çözer.
- **Parent-document (small-to-big): küçük** chunk ile ARA, ama LLM'e o chunk'ın **büyük ebeveynini** VER. Arama hassas, bağlam geniş olur.

③ Karar noktaları

Karar	Kural
Chunk boyutu	Token ile ölç. Tipik 250–500 token, modelin limitinin %80'ini geçme.
Overlap	Chunk'ın %10–20'si.
Strateji	Düz metin → recursive. Başlıklı / tablolular teknik doküman → yapı bazlı.
PDF nasıl okunur	Önce metin katmanı var mı bak. Yoksa taranmış → OCR gerekir .
Tablolar	Düz metne çevirmek satır–kolon ilişkisini bozar. Markdown tablo olarak koru.

Boyut ödünleşimi: küçük chunk = hassas arama, eksik bağlam. Büyük chunk = zengin bağlam, gürültülü arama. Parent-document yaklaşımı ikisini birleştirir.

④ Tuzaklar

- `chunk_size` karakter, model limiti token → uyumsuzluk, chunk'ın sonu sessizce silinir.
- Taranmış PDF'ten **0 chunk** çıkar, hata vermez → dosya sisteme hiç girmez. "**0 sonuç**" **asla başarı sayılmaz.**
- Overlap 0 → sınırdaki cümle bölünür, o bilgi bulunamaz.
- Metadata eklemeszen kaynak gösteremez, filtreleyemezsin.
- Tablo düzleşince sayılar yanlış satırla eşleşir → cevap kendinden emin ve yanlış olur.

Kontrol yöntemi

1. Chunk'ları **gözle oku** — cümle ortasından bölünmüş mü?
2. Her dosya için "kaç chunk çıktı" logla → **0 olana alarm kur.**
3. Token dağılımını ölç → model limitini aşan chunk var mı?

Mülakat soruları

1. Chunk boyutunu neye göre seçersin? Küçük ve büyük olmasının etkisi ne?
2. Overlap neden var?
3. Parent-document (small-to-big) retriever hangi problemi çözer?
4. Taranmış bir PDF'i nasıl fark edersin, ne yaparsın?

GÖZLEM — EZBERLEMEK İÇİN DEĞİL

Bir projede chunk ayarı 1000 **karakter**, embedding modeli ise 256 **token** kabul ediyordu → chunk'lar 400 token'a çıkıyor, 8 chunk'ın 4'ü sessizce kesiliyordu (toplam 420 token görünmez). Ayrıca taranmış bir PDF **0 chunk** üretti ve sistem bunu hata saymadı → 7 sayfalık doküman aramaya hiç girmedi.

Retrieval Kalitesi

① Bu nedir

- **Retrieval:** soruya en ilgili chunk'ları bulmak. RAG kalitesinin en belirleyici aşaması.
- **Dense arama (embedding):** anlam benzerliğine bakar. Eşanlam ve parafrazı yakalar.
- **Sparse arama (BM25):** kelime örtüşmesine bakar. Özel isim, kod, seri numarası, kısaltmayı yakalar.
- **Hybrid arama:** ikisini birlikte kullanmak. Hata modları farklı olduğu için birbirini tamamlar.
- **RRF (Reciprocal Rank Fusion):** iki sıralı listeyi **skorlarına değil sıralarına** göre birleştirme yöntemi.
- **Reranking:** adayları cross-encoder ile yeniden sıralamak. Genelde en büyük kalite artışı buradan gelir.
- **Recall:** doğru cevabı adaylar arasına alabildim mi? · **Precision:** LLM'e verdiğim az sayıda chunk gerçekten isabetli mi?

② Nasıl yapılır

```
Soru → [dense ara]  ┌
                    │→ RRF ile birleştir → cross-encoder ile rerank → en iyi 3-5 → LLM
                    └
                      → [sparse ara] ┘
```

RRF'in mantığı

Her dokümana, geçtiği her listedeki sırasına göre $1 / (60 + \text{sıra})$ puanı verilir ve puanlar toplanır. Skorlar kullanılmadığı için BM25 ile cosine'in **farklı ölçekte olması sorun olmaktan çıkar**. 60 sabiti ilk sıraların ezici üstünlüğünü kırar; böylece **iki listede birden geçen doküman**, tek listede birinci olanı geçer. RRF'in bütün değeri bu "uzlaşma ödülü"nden gelir.

Sorgu iyileştirme (retrieval'ı besleyen katman)

- **Query rewriting:** takip sorusunu ("peki maliyeti?") konuşma geçmişini kullanarak bağımsız, tam bir soruya çevir — *sonra* ara.
- **Multi-query:** sorunun 2–3 farklı ifadesini üret, hepsiyle ara, sonuçları birleştir.
- **HyDE:** LLM'e önce uydurma bir cevap yazdır, o cevabı arama sorgusu olarak kullan. Soru kısa, doküman uzunsa işe yarar.

③ Karar noktaları

Karar	Kural
Hybrid mi, tek başına dense mi?	Metinde özel isim, seri no, kod, kısaltma varsa hybrid şart . Sadece anlatı metniyse dense yeterli olabilir.
Kaç aday çekilir? (fetch_k)	20–50 çek, LLM'e 3–5 ver. Geniş tara, sonra daralt.
Reranker koyulur mu?	20+ aday varsa evet . Kalite/emek oranı en yüksek iyileştirme.
RRF mi ağırlıklı toplama mı?	RRF varsayılan — ayar (tuning) gerektirmez. Ağırlıklı yöntem her veri setinde yeniden ayarlanmak ister.
Takip soruları var mı?	Varsa query rewriting zorunlu. Aksi halde "onun fiyatı ne?" hiçbir şey bulmaz.

④ Tuzaklar

- **Birleştirme anahtarı olarak nesne kimliği kullanmak** (Python'da `id()`) → aynı içerik farklı nesnede olunca eşleşme bulunmaz, fusion sessizce çalışmaz. Anahtar **içerikten** türetilmeli (hash).
- Aynı veriyi iki kez yüklemek → nesne kimlikleri değişir, kimliğe dayanan her mantık bozulur.
- Doğrudan 3 aday çekmek → doğru chunk hiç aday olmaz; reranker onu kurtaramaz. **Recall kaybı geri alınamaz**.
- Konuşma geçmişini retriever'a vermemek → takip soruları çöker.
- BM25'i ham `split()` ile tokenize etmek → Türkçe'de büyük/küçük harf ve noktalama yüzünden eşleşme kaçır. Ayrıca Python'da `"İ".lower()` iki karakter üretir ve sessizce bozar.

Kontrol yöntemi

1. **"Kaç sonuç iki retriever'da birden geçti" sayısını logla**. Sürekli 0 ise fusion çalışmıyor demektir.
2. Bildiğin bir cevabın hangi chunk'ta olduğunu işaretle, o chunk adaylar arasına giriyor mu diye bak.
3. Sadece BM25, sadece dense ve hybrid sonuçlarını yan yana koy — hybrid gerçekten iyileştiriyor mu?

Mülakat soruları

1. Hybrid arama neden tek başına dense'ten iyi? Hangi sorgu tipinde BM25 kazanır?
2. RRF'te 60 sabiti ne işe yarar? Skor yerine sıra kullanmanın avantajı ne?
3. Neden LLM'e vereceğinden çok daha fazla aday çekersin?
4. Takip sorularını nasıl ele alırsın?

GÖZLEM — EZBERLEMEK İÇİN DEĞİL

Bir projede RRF yazılmıştı ama birleştirme anahtarı olarak nesne kimliği (`id()`) kullanılıyor, vektör store'u da iki kez yükleniyordu. Sonuç: BM25 ve dense sonuçları farklı nesnelere işaret ediyor, skorlar hiç toplanmıyordu — yani

fusion hiç çalışmıyordu, sistem fiilen sadece dense arama yapıyordu. İçerik tabanlı hash anahtarına geçilip store tek örnek yapıldığında "iki listede birden geçen chunk" sayısı 0'dan 2/3'e çıktı. Hata mesajı yoktu.

LLM & GenAI Katmanı

① Bu nedir

- **LLM:** bir sonraki token'ı tahmin eden model. "Bilgi tabanı" değil, örüntü tamamlayıcıdır — bu yüzden uydurur.
- **Token:** metnin model için en küçük parçası. Limitler ve fiyat token ile ölçülür.
- **Context window:** modelin bir seferde görebileceği toplam token (prompt + cevap).
- **Prompt:** modele verilen talimat + bağlam + soru.
- **Temperature:** rastgelelik ayarı. 0 = kararlı ve tekrarlanabilir, yüksek = yaratıcı ve değişken.
- **Hallucination:** modelin emin bir tonla uydurması.
- **Grounding:** cevabı verilen kaynağa dayandırma zorunluluğu. RAG'in varlık sebebi.
- **Structured output:** modelin serbest metin değil, belirlenen şemaya uygun JSON döndürmesi.
- **Tool calling:** modelin hangi fonksiyonu hangi argümanla çağıracağını önermesi.
- **Streaming:** cevabı token token akıtmak. Algılanan hızı belirgin artırır.

② Nasıl yapılır

RAG prompt iskeleti

```
[Rol]      Sen X dokümanları üzerinde çalışan bir asistansın.
[Kurallar] - SADECE aşağıdaki bağlamı kullan.
            - Bağlamda yoksa "Bu bilgi dokümanlarda yok" de.
            - Her iddianın yanına kaynak numarasını yaz.
[Bağlam]   [1] (kaynak: rapor.pdf s.3) ...metin...
            [2] (kaynak: rapor.pdf s.7) ...metin...
[Soru]     {kullanıcının sorusu}
```

Üç şey promptta **açıkça** yazılmalı: **(a)** sadece bağlamı kullan, **(b)** bilmiyorsan bilmediğini söyleme izni, **(c)** kaynak gösterme zorunluluğu. Üçünden biri eksikse model boşluğu uydurmayla doldurur.

Yapılandırılmış veri gerekiyorsa

Modelden serbest metin isteyip regex ile ayıklamaya çalışmak kırılgandır. Doğrusu: **JSON şeması ver, structured output kullan.** Model şemaya uymak zorunda kalır, ayrıştırma kodu ortadan kalkar.

③ Karar noktaları

Karar	Kural
RAG mi fine-tune mı?	Bilgi eklemek → RAG. Format / ton / davranış öğretmek → fine-tune. Fine-tune ile bilgi enjekte etmek pahalı, güncellenemez ve kaynak gösteremez.
Kaç chunk gönderilir?	3–5. Fazlası maliyeti artırır ve "lost in the middle" etkisiyle ortadaki bilgiyi kaybettirir.
Temperature?	Bilgi çıkarma / soru-cevap → 0–0.2 . Yaratıcı yazım → 0.7+.
Lokal mi API mi?	Veri dışarı çıkamıyorsa lokal (Ollama vb.). Kalite ve hız öncelikliyse API.
Streaming?	Kullanıcı bekliyorsa evet. Toplu işlemede gereksiz.

④ Tuzaklar

- "Sadece bağlamı kullan" demeyi atlamak → model kendi ezberinden cevap verir, kaynakla çelişir.
- "Bilmiyorum" iznini vermemek → model her koşulda bir cevap üretmek zorunda kalır ve uydurur.
- Kaynak istememek → cevap doğrulanamaz, denetlenemez hale gelir.
- Çok fazla chunk göndermek → ortadaki bilgi kaybolur, maliyet artar, kalite *düşer*.
- Konuşma geçmişini promptta koymayı unutmak → arayüzde geçmiş görünür ama model hatırlamıyordur.
- Temperature'ı yüksek bırakmak → aynı soru her seferinde farklı cevap, hata ayıklanamaz.

Kontrol yöntemi

- Negatif test:** bağlamda kesinlikle olmayan bir şey sor. Model "bilmiyorum" diyor mu, uyduruyor mu?
- Cevaptaki her iddianın gösterilen kaynakta gerçekten olup olmadığını elle doğrula.
- Aynı soruyu 3 kez sor. Cevaplar çok farklıysa temperature yüksek.

Mülakat soruları

- RAG ile fine-tune arasında nasıl seçim yaparsın?
- Hallucination'ı azaltmak için promptta hangi üç şeyi mutlaka yazarsın?
- Daha çok bağlam vermek neden kaliteyi düşürebilir?
- Modelden JSON almanın doğru yolu nedir, neden regex kullanılmaz?

Değerlendirme (Evaluation)

① Bu nedir

- **Golden set:** elle hazırlanmış *soru + doğru cevap + doğru kaynak* listesi. Değerlendirmenin temeli.
- **Recall@k:** doğru chunk, ilk k sonucun içinde mi? Retrieval'ın temel metriği.
- **MRR:** doğru sonuç kaçınıcı sıradaydı? Sıranın tersinin ortalaması. Üstte olmayı ödüllendirir.
- **nDCG:** sıralamanın genel kalitesi.
- **Faithfulness:** cevap, verilen bağlama sadık mı? (uydurma var mı)
- **Answer relevancy:** cevap soruyu gerçekten karşılıyor mu?
- **Context precision / recall:** getirilen bağlam ne kadar isabetli / ne kadar eksiksiz?
- **LLM-as-judge:** cevabı başka bir LLM'e puanlatmak. Ölçeklenir ama kusurludur.

② Nasıl yapılır

1. 30-50 soruluk golden set kur (gerçek kullanıcı sorularından türet)
2. Her soru için doğru chunk'ı elle işaretle
3. İKİ AYRI ölçüm yap:
 - a) Retrieval → Recall@k, MRR
 - b) Generation → faithfulness, answer relevancy
4. TEK bir değişkeni değiştir → tekrar ölç → karşılaştır

Sıra önemli: önce retrieval'ı ölç. Doğru chunk hiç gelmiyorsa prompt'u iyileştirmek boşa emektir — LLM elinde olmayan bilgiyi üretemez. Bu ayırım, RAG hata ayıklamanın en önemli alışkanlığıdır.

Hazır araçlar: **RAGAS**, **DeepEval** bu metrikleri hazır verir. Ama küçük bir golden set + Recall@k'yi elle hesaplamak bile hiç ölçmemekten kat kat iyidir.

③ Karar noktaları

Karar	Kural
Nereden başlanır?	Retrieval metriklerinden. Hataların çoğu orada.
Kaç soru yeter?	30–50 ile başla. Az ama gerçek sorular, çok ama uydurma sorulardan iyidir.
LLM-as-judge güvenilir mi?	Yön verici, mutlak değil. Trend takibi için iyi, tek karar mercii olarak kötü.
Ne zaman ölçülür?	Her değişiklikten önce ve sonra. Baseline olmadan "iyileştirdim" denemez.

④ Tuzaklar

- **"Gözle bakıyorum, iyi görünüyor"** → bu ölçüm değil. En yaygın ve en pahalı hata.

- Aynı anda 3 şey deęiřtirip iyileřmeyi birine atfetmek → hangisinin iře yaradıęını asla bilemezsin.
- Golden set'i sistemin kendi ıktısından üretmek → sistem kendi kendini onaylar, hatalar görünmez.
- LLM-as-judge önyargıları: **uzun** cevabı ve **ilk sıradaki** seeneęi kayırır. Sıraları karıřtır.
- Sadece cevap kalitesini ölçüp retrieval'ı ölçmemek → sorunun nerede olduęunu bulamazsın.

Kontrol yöntemi

1. Baseline skorunu **yaz ve sakla**. Her deęiřiklięi ona karřı ölç.
2. Bir deęiřiklik metrięi düşürüyorsa geri al — sezgiye deęil sayıya güven.
3. Metrik iyileřti ama kullanıcı memnun deęilse golden set'in gerek kullanımı temsil etmiyor demektir.

Mülakat soruları

1. RAG sisteminin iyileřtięini nasıl kanıtlarsın?
2. Retrieval ve generation hatalarını nasıl ayırt edersin? Hangisini önce ölçersin?
3. Recall@k ve MRR arasındaki fark ne?
4. LLM-as-judge'ın bilinen önyargıları neler, nasıl azaltırsın?

Production'a Taşıma

① Bu nedir

- **API katmanı:** sistemi HTTP servisine dönüştürmek (FastAPI). Başka uygulamaların tüketebilmesi için.
- **Bağımlılık yönetimi:** paket sürümlerini kilitlemek, böylece proje başka makinede de aynı şekilde kurulsun.
- **Gözlemlenebilirlik:** log, gecikme, token maliyeti, "hangi chunk'lar geldi" kaydı.
- **Sır yönetimi:** API anahtarlarını koddan ve istemciden ayırmak.
- **Idempotency:** aynı işi iki kez yapmamak (aynı dosyayı tekrar indekslememek).
- **Cache:** tekrar eden hesaplamayı (embedding, model yükleme) bir kez yapmak.

② Nasıl yapılır

```

UI / başka servis
  |
  | HTTP
  ▼
API (FastAPI) → retrieval → LLM → cevap + kaynaklar
  |
  |→ log: soru, dönen chunk kimlikleri, süre, token sayısı

```

- Embedding modelini ve index'i **uygulama açılışında bir kez** yükle, her istekte değil.
- `requirements.txt` gerçekten kurulu sürümleri yansıtmalı. Yansıtmazsa proje başka makinede kurulmaz.
- Cevabı **her zaman kaynaklarıyla** döndür. Kaynaksız cevap denetlenemez.
- Streaming ile ilk token'ı erken gönder — toplam süre aynı olsa bile kullanıcı deneyimi değişir.
- Kullanıcıya 👍 / 👎 koy. Bu, golden set'ini büyütecek en değerli veri kaynağıdır.

③ Karar noktaları

Karar	Kural
Streamlit mi API mi?	Hızlı demo / iç kullanım → Streamlit. Başka sistem tüketecekse veya ölçeklenecekse → API.
Ne loglanır?	Soru, dönen chunk kimlikleri, gecikme, token sayısı, kullanıcı geri bildirimi. Ham metni değil kimliği logla (gizlilik + boyut).
Sırlar nerede tutulur?	Sunucu tarafı ortam değişkeni. İstemci bundle'ına asla girmez.
İş mantığı nerede çalışır?	Sunucuda. Tarayıcıda çalışan çıkarım mantığı test edilemez, sürümlenemez, mevcut veriye yeniden uygulanamaz.

④ Tuzaklar

- **API anahtarını frontend ortam değişkenine koymak** (ör. `VITE_*`, `NEXT_PUBLIC_*`) → derlenmiş bundle'a gömülür, tarayıcıdan herkes okur. Sunucu tarafı proxy şart.
- `requirements.txt` ile kurulu sürümlerin uyuşmaması → "bende çalışıyordu" sınıfı hatalar.
- Modeli her istekte yeniden yüklemek → gecikme onlarca kat artar.
- Log tutmamak → kullanıcı "yanlış cevap verdi" dediğinde hangi chunk'ların geldiğini bilemezsin, hatayı bulamazsın.
- Pickle tabanlı index'i güvenilemeyen kaynaktan yüklemek → pickle açmak **kod çalıştırmaktır**. Sadece kendi ürettiğin index'i yükle.
- Ham metni tam olarak saklamamak (kırmak) → modeli/chunk stratejisini değiştirdiğinde yeniden işleyemezsin.

Kontrol yöntemi

1. Temiz bir sanal ortamda `pip install -r requirements.txt` çalıştır → proje kuruluyor mu?
2. Derlenmiş frontend çıktısında API anahtarını ara → bulunuyorsa sızmış demektir.
3. Bir isteğin loguna bakarak "neden bu cevabı verdi" sorusunu cevaplayabiliyor musun?

Mülakat soruları

1. API anahtarını frontend'de tutmak neden güvensiz? Doğru yaklaşım ne?
2. Bir RAG sisteminde ne loglarsın ve neden?
3. İş mantığını neden sunucu tarafında tutarsın?
4. Pickle tabanlı bir index yüklemenin riski nedir?

GÖZLEM — EZBERLEMEK İÇİN DEĞİL

Bir projede `requirements.txt` `langchain==0.1.0` yazıyordu ama kurulu sürüm **0.2.17**'ydi (`sentence-transformers` 2.3.1 yazıyordu, kurulu olan 5.1.2). Yani o dosyayla proje başka bir makinede kurulamazdı. Ayrı bir projede OCR API anahtarı `VITE_` önekli bir değişkende tutuluyordu — bu, anahtarın tarayıcıya indirilen bundle içinde açıkça yer alması anlamına gelir.